Coinductive Symmetric Homomorphism Reinforcement Learning: Optimality as Structure Preservation

Ricardo Corral-Corral

Independent Researcher, Chihuahua, Mexico
ricardo@gyx.ai

Abstract. We define a typable optimality criterion based on coinductive preservation of action dominance: an action is better than another if it leads to a strictly better future in the coinductive sense. We realize this criterion through homomorphic rankings—permutations of the action set that preserve the coinductive ordering on successor value streams.
A natural strengthening additionally requires immediate rewards to respect the ranking. These two conditions decompose cleanly; the homomorphism criterion alone is strictly more general and correctly ranks actions in sacrifice patterns where discounted return fails. We give a constructive procedure—the Finder together with violation-correction learning—that computes rankings provably satisfying the criterion. All results are machine-verified in Agda, including monotonic convergence of learning, stochastic lifting via the Giry monad, and scalable verification on large combinatorial domains with zero postulates.

Keywords: Reinforcement learning · Coinduction · Symmetric group · Formal verification · Agda · Optimality · Stochastic dominance

Code Availability. All definitions, theorems, and algorithms presented in this paper have been formalized and machine-checked in Agda 2.8.0 (Standard Library 2.0). The complete development includes the `CoinductiveHomomorphism` and `CoindHomo` records, the Finder algorithm, monotonic convergence of learning, stochastic lifting via the finite distribution monad, the Environment Class infrastructure, compositional algebra, verified state abstraction, and all verified tasks (including `Queens8`). Concrete module paths are referenced throughout the text; the full artifact is publicly available at `https://github.com/doctorcorral/csh-rl`.

1 Motivation: The Limitations of Scalar Returns

Reinforcement learning (RL) studies agents that learn to make sequential decisions by interacting with an environment [12]. At each discrete

time step, the agent observes a *state*, selects an *action*, and receives a numerical *reward* that signals how good the outcome was. The agent's goal is to find a *policy*—a rule mapping states to actions—that collects as much reward as possible over time.

The question is how to define "as much reward as possible." The dominant answer in modern RL is the *expected discounted return*: given a policy π, the agent averages over all trajectories τ it might experience:

$$J(\pi) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{\infty}\gamma^t r_t\right]$$

where r_t is the reward at time t and $\gamma \in [0,1)$ is the *discount factor*. The discount makes future rewards exponentially less valuable: a reward 10 steps ahead counts for only γ^{10} of its face value. This ensures the sum converges and makes the problem mathematically tractable.

The result is a single scalar. A policy is optimal when it maximizes this number. This formulation—rooted in Bellman's dynamic programming principle [2] and operationalized by algorithms like Q-learning [16], policy gradient methods, and deep RL [8]—has driven remarkable successes, from game playing to robotics.

1.1 The Problem: Temporal Structure Loss

Scalar return collapses the temporal structure of reward sequences. Two futures can have similar discounted sums while being structurally very different:

Future	**Reward Sequence**	**Sum (γ=0.9)**
A: "Win then die"	$[10, -5, 0, 0, \ldots]$	$10 - 4.5 = 5.5$
B: "Struggle then thrive"	$[0, 5, 0, 0, \ldots]$	$0 + 4.5 = 4.5$

A scalar optimizer prefers A, yet B is structurally superior—it avoids catastrophic failure and reaches a productive state. This is **value aliasing**: structurally different futures map to the same scalar.

This formulation introduces several artifacts:

- **Discount factor (γ):** An extrinsic parameter that undervalues delayed rewards and can make the optimal policy discontinuous in γ.
- **Instability:** Q-learning's bootstrapping of scalar estimates leads to the deadly triad (function approximation + bootstrapping + off-policy learning) [12].
- **Credit assignment:** Rewards at large t are effectively zero under discounting, crippling long-horizon learning.

These are not bugs—they follow directly from compressing sequences into scalars. When optimal behavior requires paying a cost now for a benefit later, discounting systematically distorts the decision.

1.2 Our Approach

We instead define optimality via *structure preservation*. We keep reward sequences as infinite streams and rank actions by whether they lead to successor states whose optimal futures dominate others at every depth under a coinductive ordering. The central question is what structure to preserve and how to do so formally.

2 Structural Optimality: Successor State Quality

Consider an agent choosing between two actions at a state. Action a gives an immediate reward of 3 but leads to a dead-end state (reward 0 forever). Action b gives immediate reward 0 but leads to a productive state (reward 5 forever).

A scalar return optimizer with low discount factor might prefer a. But structurally, b is the better action—it leads to a *better state*. The immediate reward is a one-time cost; the successor state quality compounds forever.

This motivates our central criterion:

> **An action is ranked higher because it leads to a successor state of higher quality.** A state is of higher quality when acting optimally from it yields higher rewards at every future depth—which itself depends on the quality of states reachable from there, unfolding coinductively.

The ranking tracks which actions lead to *success*—both in the sense of successor state and in the evaluative sense.

3 Formal Definition

3.1 The Order Homomorphism

We now make the intuition from Section 2 precise. The formalization requires three ingredients; we introduce each in turn.

Actions and rankings. Let $\mathcal{A}$ be the finite set of actions available to the agent. At each state s, the agent maintains a *ranking*—a total preorder $\leq_a$ on $\mathcal{A}$—expressing which actions it considers at least as good as which others. If $a \leq_a b$, the agent regards b as at least as good as a at state s.

The object of interest is the *entire* order on $\mathcal{A}$, not merely its maximum: a strict total ranking is precisely an element of the symmetric group $S_{|\mathcal{A}|}$ acting on the actions—one ordering among all $|\mathcal{A}|!$ permutations, whereas an argmax commitment leaves that choice almost entirely underdetermined. This full-ranking stance is foundational:

it is the sense in which the homomorphism is *symmetric*. Optimality is judged against the whole permutation structure on actions, not a single distinguished action. An agent that has genuinely understood an environment can say which of even its two worst options is less bad; an argmax-centric agent cannot. The commitment is not merely conceptual: it is what makes learning correct every violated pair, including among the worst (§14), and what makes adaptation to a lost top action an $O(1)$ lookup of the next-best (Remark 2).

This full-ranking requirement is not merely a matter of having more information. It reflects a stronger epistemic stance: an agent that only identifies its single best action has not necessarily understood the environment. Such an agent can maintain the appearance of competence for as long as its preferred action remains available; the limitations of its model only become visible when that action is lost. Worse, by never being forced to maintain coherent preferences among suboptimal actions, a top-action-only learner can enter undetected bias spirals in which its internal model of the environment is structurally faulty. Requiring every pair to respect the homomorphism forces the ranking to reflect genuine understanding rather than local preference for a single distinguished choice.

Successor states. When the agent takes action a at state s, the environment transitions to a new state and emits a reward. We write $\mathit{step}(s, a)$ for this transition, which produces a pair: the successor state $\mathit{next}(s, a)$ and the immediate reward $r(s, a)$. The successor state is where the agent *ends up*—the starting point of the rest of its life.

Value streams. From any state s', the agent can act optimally into the indefinite future, collecting a reward at each time step. Rather than collapsing this sequence into a scalar, we keep it as an infinite stream:

$$\mathit{value}(s') = [\, v_0,\ v_1,\ v_2,\ \ldots \,]$$

where v_t is the optimal reward achievable at depth t from s'. This stream retains the full temporal structure that a scalar return discards. (The precise construction is given in §4.)

Comparing streams. To say one state is "better" than another, we compare their value streams *pointwise*. We write $x \leq_s y$ when stream x is dominated by stream y at every position: for all $t = 0, 1, 2, \ldots$, the t-th element of x is $\leq_r$ the t-th element of y. This is a strong condition: the better stream must be at least as good at every single future depth, not just on average or in total.

With these ingredients in hand, the definition is a single sentence:

Definition 1 (`CoinductiveHomomorphism` — Successor Condition). *The ranking $\preceq_a$ satisfies the* ***`CoinductiveHomomorphism`*** *condition at state s if:*

$$a \preceq_a b \implies \mathit{value}(\mathit{next}(s,a)) \preceq_s \mathit{value}(\mathit{next}(s,b))$$

In words: if the ranking says b is at least as good as a, then the state reached by b must have a value stream that dominates the value stream of the state reached by a.

The definition says nothing about the immediate rewards $r(s,a)$ and $r(s,b)$. It judges actions entirely by *where they lead*. An action that pays a cost now but reaches a superior state is correctly ranked, because the successor state's value stream—reflecting the entire optimal future from that point onward—is what matters.

Why "homomorphism"? The map $a \mapsto \mathit{value}(\mathit{next}(s,a))$ sends each action to a stream, and the definition requires this map to be *order-preserving* from the action ranking into the stream ordering. An order-preserving map between ordered sets is called a *homomorphism*—it preserves the structure. The qualifier "coinductive" refers to the target: the stream ordering $\preceq_s$ unfolds to arbitrary depth, verified one position at a time, without bound—an infinite tower of commuting diagrams:

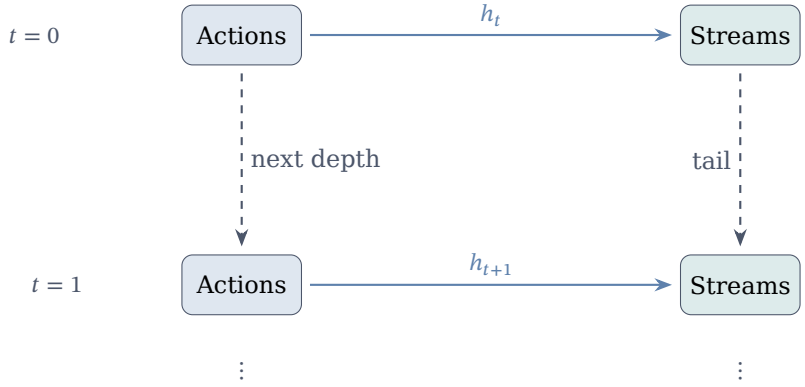


Fig. 1. The coinductive tower: h_t maps each action to its successor-value stream at depth t. Ranking preservation ($a \preceq_a b \Rightarrow h_t(a) \preceq_s h_t(b)$) must hold at every level, verified by unfolding one `tail` at a time.

This is a **strategic** evaluation of actions: it assesses the absolute potential of the successor state, allowing for immediate tactical sacrifices.

3.2 The Action-Value Specialization

The `CoinductiveHomomorphism` condition evaluates successor state quality—the agent's future *after* the transition. A natural strengthening evaluates the *entire* consequence of an action, including the immediate reward it produces.

Given action a at state s, define the *action-value stream*:

$$\textit{action-value}(s,a) = \big[\, r(s,a),\ \ v_0,\ \ v_1,\ \ v_2,\ \ \ldots \,\big]$$

Its head is the immediate transition reward; its tail is the value stream of the successor state. The action-value stream is a richer object: it records both what the action pays now and what it sets up for the future.

Definition 2 (`CoindHomo` — Action-Value Condition). *The ranking $\preceq_a$ satisfies the **`CoindHomo`** (action-value) condition at state s if:*

$$a \preceq_a b \implies \textit{action-value}(s,a) \preceq_s \textit{action-value}(s,b)$$

Because $\preceq_s$ compares streams pointwise, this bundles *two* requirements into one: the immediate rewards must compare correctly (the head), *and* the successor state qualities must compare correctly (the tail).

This is a **tactical** evaluation: it demands that the better-ranked action wins both the immediate payoff and the long-term future. When both conditions hold, the ranking is unassailable. But when the optimal action sacrifices immediate reward for a superior successor—the pattern from Section 2—the head condition breaks, and this definition cannot express the correct ranking.

3.3 Decomposition

The relationship between the two definitions is precise:

Theorem 1 (Decomposition). *The action-value condition decomposes into two independent conditions:*

1. ***Successor quality*** *(`CoinductiveHomomorphism`):* $a \preceq_a b \implies \textit{value}(\textit{next}(s,a)) \preceq_s \textit{value}(\textit{next}(s,b))$
2. ***Immediate reward compatibility****:* $a \preceq_a b \implies r(s,a) \preceq_r r(s,b)$

Either condition can hold independently. Together, they are equivalent to the action-value condition.

The first condition is `CoinductiveHomomorphism`—the strategic evaluation. The second is a purely local constraint on immediate rewards. The action-value condition is their conjunction: it insists on tactical *and* strategic soundness simultaneously.

When immediate rewards are aligned with the ranking—as they are in environments with uniform step costs, shaped rewards, or goal-terminal structure—the second condition holds for free, and the two definitions coincide. The gap opens only in environments where the optimal action has a worse immediate reward than a suboptimal alternative: the sacrifice pattern.

As sets of admissible rankings the relationship is a strict containment, `CoindHomo` ⊊ `CoinductiveHomomorphism`: uniform-step-cost environments lie in the inner set, while sacrifice-now-gain-later environments lie in the gap.

4 The Agda Core

We now present the verified implementation. Every code listing shown below is type-checked by Agda; the snippets are rendered directly from the checked source.

We chose Agda over other proof assistants (Coq, Isabelle, Lean) because the development is coinduction-heavy: Agda's copatterns and `--guardedness` discipline let stream definitions and bisimulation proofs be written in direct style, without the coinduction encodings these systems typically require. The entire development compiles under `--safe` (no postulates, no escape hatches), and literate Agda lets this paper itself be the checked artifact. Nothing in the theory is Agda-specific; ports to other assistants are possible.

The `Core` module is parameterized by the environment:

4.1 Value and Action-Value Streams

The key construction: `value` is an infinite stream obtained by tabulating finite-horizon solutions.

```
solve : State → ℕ → Reward
solve s zero = max-list (map (λ a → proj₂ (step s a)) all-actions)
solve s (suc n) = max-list (map (λ a → solve (proj₁ (step s a)) n) all-actions)

value : State → StreamR
value s = tabulate (solve s)

action-value : State → Action → StreamR
head (action-value s a) = proj₂ (step s a)
tail (action-value s a) = value (proj₁ (step s a))
```

`solve s n` computes the maximum reward achievable *exactly* n *steps from now*, independently maximizing the action at every intermediate step. Crucially, the action sequence that maximizes the reward at depth n may differ from the one that maximizes at depth m. The

resulting value stream is therefore *not a trajectory*—no single action sequence produces it. It is the pointwise upper envelope of all possible trajectories: the agent's **capability profile** from state s, recording the best it *could* achieve at each future depth.

`CoinductiveHomomorphism` compares these capability profiles. When state s_b's profile dominates s_a's at every depth, the agent prefers any action that reaches s_b—even if that action sacrifices immediate reward. The sacrifice is invisible in the profile: it only records what the agent *can* reach, not the cost of getting there.

The `action-value` of action a at state s is a stream whose head is the immediate reward and whose tail is the capability profile of the successor state.

4.2 Coinductive Stream Ordering

```
record _≤ₛ_ (x y : StreamR) : Set where
  coinductive
  field
    head≤ : head x ≤ᵣ head y
    tail≤ : tail x ≤ₛ tail y

open _≤ₛ_ public
```

A proof of $x \leq_s y$ is an infinite witness: at every time step, the heads compare correctly.

4.3 Why Propositions Instead of Booleans?

The reward ordering `_≤ᵣ_` lives in `Set`—it is a *proposition*, not a Boolean decision procedure. This is a deliberate design choice:

- **Booleans are blind:** `true` carries no information about *why* the comparison holds. To use a Boolean result in a proof, a separate soundness lemma is required.
- **Propositions carry evidence:** A value of type `r₁ ≤ᵣ r₂` *is* the proof—no lemma needed.
- **Decidability bridges both:** `Dec (r₁ ≤ᵣ r₂)` is either `yes p` (with proof `p`) or `no ¬p` (with refutation).

This separation threads through the entire architecture: the Core module uses propositions for proofs; the Finder uses Booleans for speed; the Environment Class infrastructure bridges them via `Dec`.

4.4 `CoindHomo`: The Action-Value Condition

The `CoindHomo` record is the Agda encoding of the Action-Value Coinductive Homomorphism (Definition 2):

```
record CoindHomo : Set₁ where
  field
    _≤ₐ_ : State → Action → Action → Set
    preserves : ∀ a b s → _≤ₐ_ s a b →
                action-value s a ≤ₛ action-value s b

open CoindHomo {{...}} public
```

The record has two fields. The first, $\_\leq_a\_$, is the ranking itself—a ternary relation indexed by state, taking two actions and returning a type (a proposition, per §4.3). The second, `preserves`, is the proof obligation: for any actions a, b and state s, if the ranking says $a \leq_a b$, then the full action-value stream of a is coinductively dominated by that of b. Because $\leq_s$ checks both head and tail, this bundles two requirements: the immediate reward must compare correctly (head), and the successor state quality must compare correctly (tail).

4.5 `CoinductiveHomomorphism`: The Successor Condition

The `CoinductiveHomomorphism` record is the Agda encoding of Definition 1. The successor map extracts the state reached by an action:

```
next : State → Action → State
next s a = proj₁ (step s a)

successor-value : State → Action → StreamR
successor-value s a = value (next s a)
```

`successor-value` s a is the optimal value stream from the state reached by taking action a at state s. It is definitionally equal to $\mathit{tail}(\mathit{action\text{-}value}\ s\ a)$.

```
record CoinductiveHomomorphism : Set₁ where
  field
    _≤ₐ_ : State → Action → Action → Set
    preserves : ∀ a b s → _≤ₐ_ s a b →
                successor-value s a ≤ₛ successor-value s b
```

The `CoinductiveHomomorphism` condition requires only that the ranking preserves successor state quality. The immediate transition reward is decoupled by design.

4.6 The Decomposition

```
CoindHomo→CoinductiveHomomorphism : CoindHomo → CoinductiveHomomorphism
CoindHomo→CoinductiveHomomorphism homo = record
  { _≤ₐ_ = CoindHomo._≤ₐ_ homo
  ; preserves = λ a b s prf → tail≤ (CoindHomo.preserves homo a b s prf)
  }
```

Every `CoindHomo` is a `CoinductiveHomomorphism`: project the tail of the action-value stream dominance. The converse does *not* hold.

```
CoinductiveHomomorphism+Head→CoindHomo :
  (ch : CoinductiveHomomorphism) →
  (∀ a b s → CoinductiveHomomorphism._≤ₐ_ ch s a b →
    proj₂ (step s a) ≤ᵣ proj₂ (step s b)) →
  CoindHomo
CoinductiveHomomorphism+Head→CoindHomo ch head-compat = record
  { _≤ₐ_ = CoinductiveHomomorphism._≤ₐ_ ch
  ; preserves = λ a b s prf →
      combine-≤ₛ (head-compat a b s prf)
                 (CoinductiveHomomorphism.preserves ch a b s prf)
  }
```

When immediate transition rewards also respect the ranking, `CoinductiveHomomorphism` specializes to `CoindHomo`. This characterizes exactly what `CoindHomo` adds: the head constraint, which is orthogonal to successor state quality.

5 When the Conditions Coincide—and When They Diverge

The decomposition theorem reveals that the two conditions differ by exactly one requirement: whether immediate transition rewards respect the ranking. When does this matter?

5.1 When the Two Conditions Coincide

In many standard environments the head condition holds for free, so `CoindHomo` and `CoinductiveHomomorphism` coincide. This includes environments with uniform step costs, shaped rewards that already guide the agent toward optimality, and goal-terminal settings where the only distinction is reaching the goal versus not. In these cases every `CoinductiveHomomorphism` is automatically a `CoindHomo`, and all results proved under `CoindHomo` apply directly.

5.2 When They Diverge: The Sacrifice Pattern

The two conditions diverge precisely when the optimal action has a *worse* immediate reward than a suboptimal alternative. We call this the **sacrifice pattern**: the agent must accept an immediate cost to reach a superior successor state.

This pattern is invisible both to discounted return (which suppresses future payoffs geometrically) and to `CoindHomo` (which refuses to rank when the head condition fails). The `CoinductiveHomomorphism` condition is the first in the framework that fully decouples optimality from immediate reward, judging actions solely by the quality of the states they lead to.

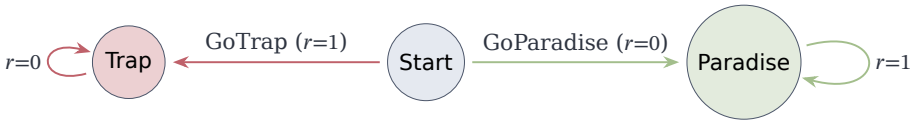
Most standard benchmarks have been shaped so that immediate rewards already align with long-term quality. Environments where immediate rewards actively mislead are regarded as "hard exploration problems" and are typically studied as pathological cases rather than representative ones. As a result, the sacrifice pattern—where the greedy action is wrong—has been quietly engineered out of the environments the field studies most. `CoindHomo` remains the right tool for the common (aligned) case; `CoinductiveHomomorphism` extends the framework to the complementary regime without losing anything that `CoindHomo` provides.

6 Strict Generality: Verified Counterexamples

The implication `CoindHomo` ⇒ `CoinductiveHomomorphism` is strict. We prove this with three verified environments of increasing structural complexity, each exhibiting the sacrifice pattern.

6.1 Sacrifice Now, Gain Later

The minimal separating example has three states and two actions:



GoParadise sacrifices the immediate reward (0 vs 1) but leads to Paradise (reward 1 forever). GoTrap grabs the immediate reward but leads to Trap (reward 0 forever).

`CoinductiveHomomorphism`: $\mathit{value}(\text{Trap}) = [0, 0, \ldots] \leq_s [1, 1, \ldots] = \mathit{value}(\text{Paradise})$ □

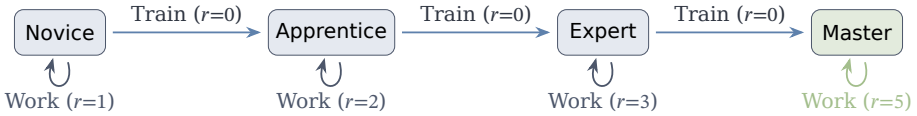
`CoindHomo`: Cannot rank GoTrap ≤ GoParadise (head requires $1 \leq 0$). Cannot rank GoParadise ≤ GoTrap either (tail comparison yields $1 \leq 0$

at the head of the successor streams). The actions are *incomparable*—even though one is clearly better.

Full proof: `src/CSHRL/Tasks/Verified/BinarySacrifice.agda`

6.2 Skill Investment: Multi-Stage Sacrifice

A chain of investment decisions where training sacrifices immediate reward to advance:



The capability profiles (value streams) form a clean progression:

$$\begin{aligned} \text{Master:} &\quad [5, 5, 5, 5, \ldots] \\ \text{Expert:} &\quad [3, 5, 5, 5, \ldots] \\ \text{Apprentice:} &\quad [2, 3, 5, 5, \ldots] \\ \text{Novice:} &\quad [1, 2, 3, 5, \ldots] \end{aligned}$$

These are *not trajectories*. Consider the Apprentice profile $[2, 3, 5, \ldots]$: the 2 at depth 0 comes from Working (best immediate reward), the 3 at depth 1 comes from Training to Expert *then* Working (a different action plan), and the 5 at depth 2 comes from Training twice to Master then Working (yet another plan). Each entry independently records the best the agent *could* achieve at that depth. No single action sequence produces the stream $[2, 3, 5, \ldots]$.

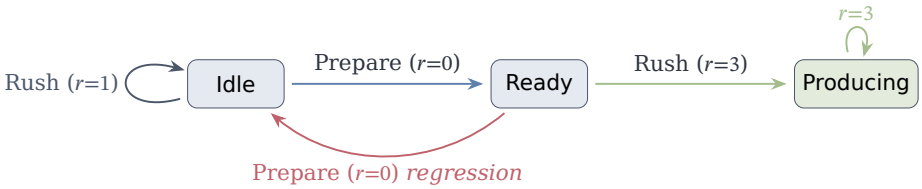
`CoinductiveHomomorphism` compares these profiles: Train's successor (Expert, profile $[3, 5, 5, \ldots]$) pointwise dominates Work's successor (Apprentice, profile $[2, 3, 5, \ldots]$). The sacrifice—Train's immediate reward of 0 versus Work's 2—does not appear in the successor profiles, because the profiles record reachable capability, not incurred cost.

`CoindHomo` fails at every level: Work's immediate reward always exceeds Train's, so the action-value streams are pointwise incomparable.

Full proof: `src/CSHRL/Tasks/Verified/SkillInvestment.agda`

6.3 Preparation Dilemma: Cyclic Dynamics

A structurally different pattern with regression and context-dependent optimal actions:



At Idle, the optimal action is Prepare (sacrifice now for readiness). At Ready, the optimal action flips to Rush (capitalize on preparation). Over-preparing at Ready sends the agent *back* to Idle—a regression penalty.

Value streams: Producing = Ready = $[3, 3, 3, \ldots]$, Idle = $[1, 3, 3, \ldots]$.

`CoinductiveHomomorphism` handles the context-dependent ranking: Prepare > Rush at Idle (successor quality improves), Rush > Prepare at Ready (successor quality improves).

`CoindHomo` cannot rank the actions at Idle in *either* direction:

- Forward (Rush ≤ Prepare): head requires $1 \leq 0$—absurd.
- Reverse (Prepare ≤ Rush): head is $0 \leq 1$ (ok), but the tail requires $value(\text{Ready}) \leq_s value(\text{Idle})$, which fails at the head: $3 \leq 1$—absurd.

Full proof: `src/CSHRL/Tasks/Verified/PreparationDilemma.agda`

7 Relationship to Classical Reinforcement Learning

The value streams compared by `CoinductiveHomomorphism` are computed from the actual environment dynamics by the `solve` function—they are ground truth, not estimates. If the environment terminates (absorbing states with zero reward), the value streams reflect that. If the environment is infinite, the streams extend accordingly. The horizon is not a parameter of the condition; it is encoded in the dynamics.

7.1 Relationship to Classical Optimality

Both conditions are sound with respect to classical objectives, and each subsumes a different one.

`CoindHomo` subsumes discounted return. The action-value condition compares the *full* action-value streams, whose head is the immediate reward. Pointwise dominance at every position implies that the cumulative sum—at every finite horizon—is at least as large for the better-ranked action. Since discounted return is a positively weighted sum, action-value dominance implies discounted-return dominance for every $\gamma \in [0, 1)$: any policy optimal under `CoindHomo` is optimal under every discounted-return criterion.

`CoinductiveHomomorphism` subsumes successor return. The successor condition compares successor value streams—the accumulated reward *from the state the action leads to*. The verified theorem: The top-ranked action leads to the state with the highest accumulated reward at every finite horizon. This is successor return dominance: the better-ranked action places the agent in a strictly better position.

7.2 `CoinductiveHomomorphism` Is Sound Where Classical Methods Can Fail

`CoindHomo` refuses to rank in sacrifice environments, remaining conservative. By contrast, the `CoinductiveHomomorphism` condition continues to produce correct rankings, while classical discounted methods can actively select the *wrong* action in exactly these cases. We demonstrate this concretely with Q-learning on the binary sacrifice MDP.

Q-learning maintains a table of action-values $Q(s, a)$ estimating expected discounted return and selects $\arg\max_a Q(s, a)$. On the binary sacrifice MDP the converged values at Start are:

$$Q(\text{Start}, \text{GoTrap}) = 1 + \gamma \cdot 0 = 1$$

$$Q(\text{Start}, \text{GoParadise}) = 0 + \gamma \cdot \frac{1}{1-\gamma} = \frac{\gamma}{1-\gamma}$$

For $\gamma < 1/2$ we have $\gamma/(1-\gamma) < 1$, so Q-learning selects GoTrap—the action whose successor state has accumulated reward 0 at every horizon—over GoParadise, whose successor has accumulated reward N at horizon N. We prove in Agda (`src/CSHRL/Analysis/QLearningFailure.agda`) the conjunction of two facts:

1. **Tactical:** GoTrap has the higher immediate reward.
2. **Strategic:** For every horizon N, the accumulated reward from GoParadise's successor strictly exceeds that of GoTrap.

Together: for $\gamma < 1/2$, Q-learning selects the action with the higher immediate reward and the strictly inferior successor state. The discount factor geometrically suppresses the evidence of GoParadise's superiority until it is no longer visible to the algorithm.

Deeper sacrifice chains. The Skill Investment chain sharpens the point. Q-learning requires $\gamma > (1/5)^{1/3} \approx 0.585$ to identify the optimal training path. Below this threshold, it converges to the policy of working at Novice forever—earning 1 per step while a three-step investment would yield 5 per step permanently. `CoinductiveHomomorphism` identifies the training path regardless: the successor state quality is strictly better at every depth, and the environment dynamics confirm it.

7.3 `CoindHomo` as the Conservative Special Case

`CoindHomo` is best understood as a deliberate conservative restriction of the `CoinductiveHomomorphism` condition. One might prefer it precisely when its extra demand—that immediate rewards also respect the ranking—is desirable: it bundles tactical and strategic soundness, so its rankings are compatible with existing discounted-return theory without further argument. In aligned environments (Section 5), where the two conditions coincide, this comes for free, and `CoinductiveHomomorphism` inherits `CoindHomo`'s full subsumption of classical RL.

The restriction has a cost. In sacrifice environments `CoindHomo` cannot rank the actions at all—the action-value streams are pointwise incomparable—whereas `CoinductiveHomomorphism` still identifies the better successor state, which the environment dynamics validate. `CoindHomo` refuses to rank when immediate reward conflicts with the future; `CoinductiveHomomorphism` resolves the conflict in favor of successor state quality.

8 Stream Isomorphism

We defined stream dominance coinductively: $x \leq_s y$ holds if the heads compare and the tails recursively compare. This definition is equivalent to pointwise dominance at every timestep. The forward direction follows by unfolding the definition. The converse is established by coinduction: if $x_n \leq_r y_n$ for every n, then both `head≤` and `tail≤` hold, hence $x \leq_s y$.

This equivalence has two direct consequences. First, it confirms that the coinductive formulation precisely captures dominance at every future depth. Second, it justifies the completeness of the Finder: because the Finder decides rankings via finite trace comparison, and trace dominance at all depths implies coinductive stream dominance, every ranking returned by the Finder satisfies the coinductive condition.

The isomorphism holds for both successor-value and action-value streams, and therefore supports both `CoinductiveHomomorphism` and `CoindHomo`. The full proof is in `src/appendix/StreamIsomorphism.agda`.

9 The Finder Algorithm

The preceding sections established what an optimal ranking *is*. We now turn to how to *compute* one. The Finder algorithm uses lexicographic trace comparison and requires no discount factor.

9.1 Lexicographic Comparison

Compare traces element-by-element—the first difference decides: Unlike summation (which discards timing information), lexicographic comparison preserves it: a trace $[10, -5]$ dominates $[0, 5]$ because $10 > 0$ at step 0, regardless of subsequent elements.

9.2 Trace Evaluation

Given the comparator, we need the traces themselves. For a deterministic MDP, the trace of action a at state s to depth k is the finite prefix of the reward stream: take action a, record the reward, then act optimally (according to all actions, greedily at each step) for the remaining $k-1$ steps. `best-trace` computes this optimal continuation; `trace-action` prepends the immediate reward of a specific action:

9.3 The Ranking Finder

With traces and a comparator in hand, producing a ranking is straightforward: compute each action's trace at the desired depth, then sort by lexicographic dominance. The head of the sorted list is the optimal action; the full list is the total ranking used by the adaptation layer (Remark 2).

No discount factor is required. Standard RL needs $\gamma < 1$ to ensure convergence of the sum; here we compare structure directly, and the lexicographic order is well-defined at any depth.

9.4 Serving Both Conditions

The Finder as defined above computes `trace-action`—whose head is the immediate reward and whose tail is the optimal future. This directly serves **`CoindHomo`**: action-value traces are the finite approximations of action-value streams.

For **`CoinductiveHomomorphism`**, the relevant comparison is between *successor-value traces*—the optimal future from the state reached by each action, *excluding* the immediate reward. The adaptation is a one-line change: instead of `r :: future`, use `future` alone. Equivalently, compare `tail(trace-action s a k)` for each action.

In aligned environments (Section 5), both comparisons yield the same ranking. In sacrifice environments, the action-value trace may be incomparable (the head reversal prevents lexicographic dominance), while the successor-value trace correctly identifies the better successor state. The Stream Isomorphism (§8) guarantees that trace dominance at all depths implies coinductive stream dominance—for whichever stream variant is used.

10 Environment Classes: Modularity Without Postulates

The Finder produces rankings; the Stream Isomorphism connects them to coinductive stream dominance. But each task still needs its own bridge lemma—a potentially tedious and error-prone process. The **Environment Class** (EC) abstraction eliminates this boilerplate. Rather than proving preservation from scratch for each task, we factor out common structure into reusable modules.

10.1 Parameterized Modules

Without environment classes, every task would need its own Finder, trace-to-stream bridge, and preservation skeleton—duplication that also raises trust questions about which parts are verified. Environment classes instead provide **verified infrastructure** that task instances inherit: a task author fills in a parameter record (`ECParameters`: types, step function, reward order with decidability and extrema, and finiteness witnesses), and the class automatically supplies the Finder, trace types, stream bridges, and proof scaffolding (full implementation in `src/CSHRL/EnvironmentClass/`).

The fields divide into a *domain* group (`State`, `Action`, `step`), an *algebraic* group (the ordered reward structure $\_\leq_r\_$, $\_\leq?\_$, `max`, `bottom`), and an *enumeration* group (`all-actions`, `horizon`) supplying the finiteness witnesses the Finder needs. From these, the EC derives the trace orderings, the Finder, the ranking relation, the trace-to-stream bridge, and—for specialised ECs—a full `CoindHomo`.

10.2 What Task Instances Must Provide

For the general **FiniteDeterministicMDP**, a task supplies the domain (`State`, `Action`, `step`) plus a direct proof that the Finder's ranking preserves action-value ordering. The specialised **CombinatorialPlacementMDP** (for placement problems such as N-Queens, Sudoku, Graph Coloring) reduces this burden further; full implementation in `src/CSHRL/EnvironmentClass/CombinatorialPlacementMDP.agda`.

The three constructors model a placement problem's life cycle: `Ongoing` wraps an active configuration, `Dead` is an absorbing failure (stream $[0, 0, \ldots]$), and `Solved` a completed configuration (stream $[R, R, \ldots]$). Any path reaching Solved thus dominates any path reaching Dead at every timestep—exactly the coinductive order's requirement. The EC factors preservation into three reusable layers—`WithAbsorbingLemmas` (absorbing-state domination), `WithBinaryStructure` (every value is 0 or R, with monotonicity), and `WithTraceBridge` (horizon sufficiency $\Rightarrow$ full `CoindHomo`)—so the task author supplies at most four simple ingredients while the machinery handles the finite-trace-to-infinite-stream bridge.

Validation: 8-Queens. The `Queens8` module (`src/CSHRL/Tasks/Verified/Queens8.agda`) instantiates the EC with `--safe` and zero postulates: discharging `solve-horizon-suf` and opening `WithTraceBridge` yields a complete `CoindHomo` over a combinatorial search space ($8^8 = 16{,}777{,}216$ candidate placements) where direct stream enumeration is infeasible. More broadly, verified tasks (TwoState, Queens1, Queens8) carry no postulates; classic tasks may postulate hard-to-formalise properties while keeping the *structural* preservation proof verified; and new tasks inherit all machinery by instantiating the parameter record.

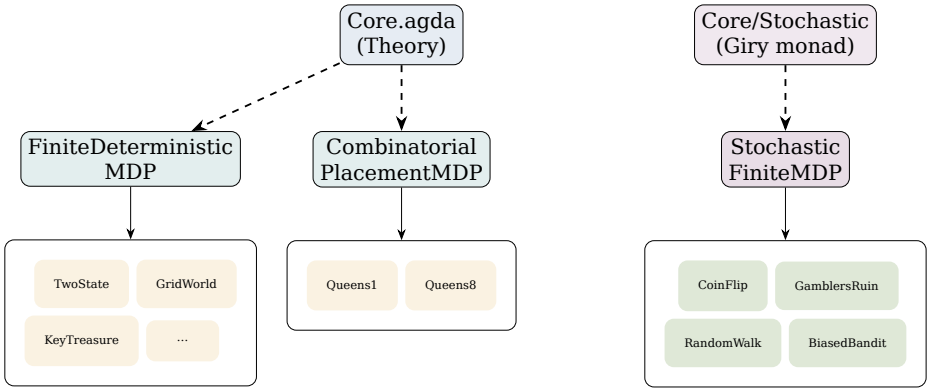


Fig. 2. Environment class hierarchy. Core provides theory; ECs provide reusable verification infrastructure; task groups contain domain-specific instantiations.

11 Stochastic Extension

Everything up to this point assumes deterministic environments. Many real-world problems are inherently stochastic: a robot's actuators have noise, a card game involves shuffled decks, or a medical treatment has variable patient responses. In such environments the step function returns a probability distribution over outcomes rather than a single successor state and reward.

The classical approach to stochastic MDPs uses the Giry monad [5], but its full generality (measurable spaces and σ-algebras) is far too heavy for constructive, decidable verification in Agda. We therefore use a *finite distribution monad*: a discrete, constructive approximation that represents distributions as weighted lists. This gives exact arithmetic, decidable equality, and compatibility with Agda's termination checker while remaining expressive enough for any finite-state

stochastic MDP. The monad operations (`pure`, `scale`, `>>=`) and their laws are verified in `src/CSHRL/Probability/Finite.agda`.

11.1 Lexicographic Coinductive Comparison

In stochastic MDPs the value at each depth is an *expected* reward. The deterministic pointwise order $\leq_s$ is too strict here: if action a already wins at depth 0 in expectation, demanding it also wins at every later depth would rule out many valid rankings. The appropriate notion is *lexicographic coinductive comparison* ($\leq_{lex}$): compare expected heads, and recurse on the tails only when the heads are equal. This remains coinductive but lets earlier rewards break ties.

11.2 Stochastic Coinductive Homomorphism

We can now lift both conditions from the deterministic setting.

- **StochasticCoindHomo** preserves expected action-value streams lexicographically. This bundles the immediate expected reward and the expected successor quality.
- **StochasticCoinductiveHomomorphism** preserves expected successor-value streams lexicographically. This decouples immediate reward, allowing sacrifice-pattern reasoning in stochastic environments.

The decomposition theorem (Theorem 1) lifts directly: the stochastic action-value condition decomposes into expected-immediate-reward compatibility plus stochastic successor-quality preservation. The full implementation is in `src/CSHRL/Core/Stochastic.agda`, with the corresponding Environment Class in `src/CSHRL/EnvironmentClass/StochasticFiniteMDP.agda`.

All four verified stochastic examples (`CoinFlip`, `GamblersRuin`, `RandomWalk`, `BiasedBandit`) follow the same verification pattern: one action dominates in expected immediate reward (so the head obligation follows from the ranking hypothesis), while the tail obligation requires two expected heads to be equal—an absurdity discharged by the empty pattern. All four are therefore fully verified with zero postulates.

11.3 Distributional Hierarchy

The framework so far compares actions by their expected reward streams. Stronger distributional guarantees are available through a hierarchy of stochastic dominance relations. **First-Order Stochastic Dominance** (FOSD) requires that the cumulative distribution of one action lies everywhere below that of another at every timestep. The

Stochastic Dominance Hierarchy generalizes this: SD[0] = FOSD, SD[1] = SOSD, SD[2] = TOSD, etc. Each additional level integrates the CDF and captures a broader class of utility functions while remaining verifiable.

A single FOSD verification automatically yields optimality at all higher SD levels. The complete pipeline (distribution level and stream level) is machine-verified in `src/CSHRL/Probability/FOSD.agda`, `src/CSHRL/Core/FOSD.agda`, `src/CSHRL/Probability/SD.agda`, and `src/CSHRL/Core/SD.agda`. These stronger distributional conditions align naturally with `StochasticCoindHomo`; the lexicographic `StochasticCoinductiveHomomorphism` sits outside the per-timestep hierarchy, as expected.

Pointwise dominance implies lexicographic dominance (but not conversely), and the two coincide in the deterministic case. Classical RL is subsumed via linearity of expectation.

12 Composition for Independent Subsystems

When an environment consists of multiple truly independent components (no shared state and no interaction between their transitions), verified rankings on each component can be combined without additional proof effort. This is useful in settings where a system is naturally modular, such as separate control loops or disjoint subsystems.

The foundation is that the SD[k] ordering is closed under distribution concatenation (`SD-++`) and non-negative scaling (`SD-scale`). Both closures are one-line consequences of monotonicity. A `VerifiedRanking` packages a ranking together with a machine-checked proof that it preserves SD[k] at every state and timestep. The algebra then provides four verified operations that combine such rankings with zero extra proof burden:

- **Product** (`++-product`): combine rankings over the product of two independent MDPs.
- **Convolution** (`conv-product`): combine rankings over additive-reward products via discrete Abel summation.
- **Sum**: combine rankings over disjoint environments.
- **Scaling**: scale a verified ranking by a non-negative factor.

These operations are implemented and verified in `src/CSHRL/Probability/Compose.agda` and `src/CSHRL/Core/Compose.agda`. They allow modular verification when the environment genuinely factors into independent parts. They do not apply to tasks whose components interact, nor do they provide a general decomposition technique for monolithic MDPs.

The same compositional infrastructure lifts to the stochastic setting and is used by the verified stochastic tasks.

13 Verified State Abstraction

Everything so far applies to finite MDPs where the state space can be enumerated. But many real-world problems have infinite or very large state spaces—a robot's continuous position, a game with billions of configurations. Verified state abstraction bridges this gap: verify a ranking on a small, finite *abstract* system, then lift it to the full concrete system.

A `StateAbstraction` bundles three components:

- **project**: Concrete → Abstract—collapses concrete states to abstract classes.
- **embed**: Abstract → Concrete—selects a representative for each class.
- **section**: `project` ∘ `embed` = id—the representatives are consistent.

No finiteness assumption is placed on `Concrete`: it can be $\mathbb{N}$, $\mathbb{R}^n$, or any type in `Set`.

13.1 The Lifting Theorem

The central result is `abstract-lift`:

> *Given a state abstraction and a proof that the marginal reward function is constant within each abstract class (marginal-invariance), any `VerifiedRanking` on the abstract system lifts to a `VerifiedRanking` on the full concrete system.*

This is the standard model-irrelevance condition from the state abstraction literature [7], but formalized constructively in Agda with a machine-checked proof. The verification cost is $O(|\text{Abstract}|)$ regardless of the concrete space's cardinality.

Three composition operators extend the framework: `product-abstraction` (component-wise), `compose-abstraction` (chaining two-level abstractions $C \xrightarrow{\alpha_1} M \xrightarrow{\alpha_2} A$), and `abstract-conv-product` (the full pipeline: abstract each component, verify FOSD abstractly, compose via convolution, lift to the concrete product). Validated postulate-free on infinite-state environments—Regional Policy ($\mathbb{N}$ states), Three-Zone ($\mathbb{N}$ states), and CartPole ($\mathbb{N}^4$ discretized, angle-based abstraction). Full implementation: `src/CSHRL/Core/Abstraction.agda`.

14 Learning as Symmetry Restoration

The Finder computes rankings at a fixed depth. Real learning, however, is incremental: the agent observes samples, detects violations of the homomorphism condition, and restores them.

A *violation* occurs when the current ranking at a state disagrees with the ground-truth comparator (instantiated by the Finder at the current depth). The repair is a single transposition: swap the two actions so the better one precedes the worse one. Each swap fixes exactly one violated pair without disturbing unrelated pairs. Because transpositions generate the symmetric group $S_{|\mathcal{A}|}$ (§3), repeated swapping corresponds to walking through the space of total rankings until the environment's stream order is restored.

Formally, a violation records the state, the pair of actions, and the depth at which the disagreement was found. The fix is to apply the transposition in the ranking.

Theorem 2 (Swap Monotonicity). *Let v be a violation with actions $(better, worse)$. After swapping: (1) the violated pair is correctly ordered; (2) unrelated pairs are unchanged; (3) the total number of violations decreases by at least* 1*.*

Since each swap strictly decreases the violation count and the count is non-negative, the process terminates. The worst-case bound is

$$\text{max-violations} \leq |S| \times \binom{|\mathcal{A}|}{2} = |S| \times \frac{|\mathcal{A}|(|\mathcal{A}|-1)}{2}.$$

The `ConvergenceTheorem` packages the full guarantee: given a valid violation finder and a sound ground-truth oracle, iterated swapping from any initial ranking reaches zero violations in at most `max-violations` steps. All proofs are in `src/CSHRL/Learning/Base.agda`.

Remark 1 (Full-ranking learning dissolves the exploration/exploitation trade-off). The convergence bound counts *all* pairwise violations, including those between the two worst actions. Uniform pairwise sampling therefore suffices for convergence; no separate exploration mechanism is required.

The same learning loop applies to both `CoinductiveHomomorphism` and `CoindHomo` (the only difference is whether the oracle compares successor-value or action-value traces). The output is a total ranking at every state.

The learning infrastructure extends to stochastic MDPs (comparing expected traces) and to `CombinatorialPlacementMDP` (short-circuiting traces for absorbing states). All monotonicity guarantees carry over.

Remark 2 (Reliable adaptation under action unavailability). Because the learned ranking is a total preorder that satisfies the homomorphism property at every state, the relative order among any pair of actions remains valid even after some actions are removed. Consequently, when the top-ranked action becomes unavailable, the second-ranked action is already known to be the best among those that

remain—an $O(1)$ lookup. The core preservation step under demotion is machine-checked (`demote-preserves-dominance` in `src/CSHRL/Learning/Base.agda`). This holds for both `CoinductiveHomomorphism` and `CoindHomo` without retraining or re-verification.

15 Discussion: Strengths and Limitations

Strengths. The framework eliminates the discount factor as an optimality requirement, replacing it with pointwise stream dominance, and it correctly ranks sacrifice patterns—verified in `BinarySacrifice`, `SkillInvestment`, and `PreparationDilemma` (§6)—where discounted return provably fails (§7). Every result is machine-checked in Agda under `--safe` with zero postulates, so there are no unstated assumptions. The full-ranking stance yields concrete operational benefits: convergent learning without a separate exploration mechanism (Remark 1) and $O(1)$ adaptation to action unavailability (Remark 2).

The ideal versus the machinery. It is important to distinguish the *coinductive ideal* from the *algorithmic machinery*. The definitions in §4 are about infinite streams and pointwise dominance at every time step; the `CoinductiveHomomorphism` condition is undecidable in general. We view this as analogous to AIXI [6]: an uncomputable ideal that organizes a hierarchy of practical approximations. The Finder (§9) compares *finite* traces at a fixed depth and becomes exact under conditions verified by the Environment Class infrastructure (e.g., horizon sufficiency); the Learner (§14) wraps Finder-style comparisons into a learning loop. Crucially, the full condition *is* achievable for structured domains: the 8-Queens instance attains it by exploiting binary reward structure and finite game depth.

Limitations. The deterministic framework assumes a finite action space, a deterministic step function, decidable reward order, and a horizon parameter for the Finder approximation (Section 11 lifts to stochastic environments; Section 13 to infinite state spaces). Scaling to *continuous* action spaces remains open (§18). Because computing exact traces requires access to the step function, the current concrete instantiations sit closer to model-based RL and formal planning than to pure model-free RL; the learning loop itself, however, only consumes pairwise comparisons and is agnostic to how the oracle obtains them. Finally, this paper's contribution is foundational and its validation is by machine-checked instantiation rather than empirical benchmark; an empirical evaluation of the heuristic, full-ranking synthesis this theory licenses—in robotics and safety-critical control, where verified fallback rankings under actuator failure are directly valuable—is ongoing work beyond the scope of this paper.

Table 1. Verified instantiations across the three environment classes.

Task	Environment class	Phenomenon demonstrated
TwoState	FiniteDeterministicMDP	Minimal aligned case; postulate-free
Delayed Gratification	FiniteDeterministicMDP	Depth-discovery (depth 2 needed)
Trap Sparse	FiniteDeterministicMDP	Discrimination at depth 1
Key-Door-Treasure	FiniteDeterministicMDP	State-dependent ranking flip
8-Queens	CombinatorialPlacementMDP	Scalable verification (8^8); postulate-free
BinarySacrifice, SkillInvestment, PreparationDilemma	FiniteDeterministicMDP	Separate the two conditions (§6)
CoinFlip, GamblersRuin, RandomWalk, BiasedBandit	StochasticFiniteMDP	Lexicographic stochastic verification; postulate-free

16 Verified Instantiations: From Theory to Tasks

The examples in this section validate that the formal machinery instantiates correctly. Each machine-checked development is a proof that the framework applies to that task class; Table 1 summarizes the verified instantiations across the three environment classes.

All tasks are machine-checked. Classic tasks use the general `FiniteDeterministicMDP` class, while structurally specialised classes (`CombinatorialPlacementMDP`, `StochasticFiniteMDP`) discharge the finite-trace-to-infinite-stream bridge automatically.

TwoState and Key Phenomena. `TwoState` is the minimal working example: at `Start`, `Go`'s future stream is $(1, 1, 1, \dots)$ and `Stay`'s is $(0, 0, 0, \dots)$; the ranking $[\texttt{Go}, \texttt{Stay}]$ satisfies both conditions. Full proof in `src/CSHRL/Tasks/Verified/TwoState.agda` (no postulates).

Beyond this base case, the tasks isolate distinct phenomena:

- `DelayedGratification` requires depth 2 to separate two paths with equal immediate reward.
- `TrapSparse` resolves at depth 1.
- `Key-Door-Treasure` exhibits a state-dependent ranking flip (phase transition encoded directly in the traces).

Scalable Verification: 8-Queens. The `Queens8Learning` module validates the full pipeline end-to-end: learning via `materialize-on-path`

from the empty board produces a `PolicyTable` of eight entries. The materialized policy yields a valid non-attacking 8-Queens placement. The complete development is in `src/CSHRL/Tasks/Verified/Queens8.agda` and runs with `--safe` and zero postulates over a search space of $8^8 = 16{,}777{,}216$ candidate placements.

Sacrifice Environments: Where the Conditions Separate. The three environments from §6—`BinarySacrifice`, `SkillInvestment`, and `PreparationDilemma`—are the verified instantiations that separate the two conditions. Each has a machine-checked `CoinductiveHomomorphism` proof together with a machine-checked proof that no `CoindHomo` can rank the sacrificing actions. These live in:

- `src/CSHRL/Tasks/Verified/BinarySacrifice.agda`
- `src/CSHRL/Tasks/Verified/SkillInvestment.agda`
- `src/CSHRL/Tasks/Verified/PreparationDilemma.agda`

All four stochastic examples (`CoinFlip`, `GamblersRuin`, `RandomWalk`, `BiasedBandit`) follow the same verification pattern and are fully verified with zero postulates in `src/CSHRL/Tasks/Verified/`.

17 Related Work

Our framework sits at the intersection of several lines of research in reinforcement learning and formal methods. We focus here on the most directly related areas and highlight where our approach differs in ways that matter for adoption.

17.1 Lexicographic Multi-Objective RL

Lexicographic multi-objective RL (LMORL) [11,13] orders multiple reward signals lexicographically. Our framework also uses lexicographic comparison, but applies it across *timesteps* of a single reward signal rather than across objectives. This makes our approach applicable even in single-objective settings where the temporal structure of reward matters. Because our ordering is coinductive and machine-checked, it provides stronger guarantees than typical LMORL formulations, at the cost of requiring a finite action space and decidable reward order.

17.2 Handling Misalignment Between Immediate and Long-Term Reward

Environments in which the optimal action has worse immediate reward than a suboptimal alternative (the “sacrifice pattern”) are well-known

but usually treated as pathological. Classical discounted RL addresses them by increasing γ or via reward shaping; both are indirect and can introduce new distortions. Safe and constrained RL [4] typically encode long-term requirements as hard constraints rather than as part of the optimality criterion itself.

Our approach is different: `CoinductiveHomomorphism` makes successor-state quality the sole determinant of action ranking, cleanly decoupling strategic evaluation from immediate reward. This is the first formal criterion we are aware of that directly ranks actions by the coinductive dominance of the states they lead to, without requiring the immediate reward to align.

17.3 Preference-Based and Ordinal Reinforcement Learning

A growing body of work learns from pairwise preferences rather than scalar rewards [17], most prominently through RLHF and Direct Preference Optimization (DPO) [9]. These methods typically extract an implicit scalar reward from preference data and then optimize it. Our framework takes the opposite stance: the pairwise ordering *is* the optimality criterion, preserved coinductively across infinite horizons. Because we maintain a total ranking over the entire action set (an element of the symmetric group), downstream tasks such as adaptation to action unavailability become $O(1)$ lookups rather than requiring policy recomputation.

Deep ordinal reinforcement learning [18] modifies Q-learning for ordinal rewards via thresholded updates. While it retains discounting and bootstrapping, it shares with our approach the recognition that ordinal comparisons can be more natural than scalar targets in some settings.

17.4 Distributional Reinforcement Learning

Distributional RL [1,3] replaces scalar value estimates with distributions over returns, enabling richer risk-sensitive and multi-modal reasoning. Our stochastic extension (§11) also moves beyond expectations by maintaining a hierarchy of stochastic dominance relations (FOSD, SD[k]). The key difference is that we compare *streams* of distributional information coinductively rather than summarizing them into return distributions. This preserves temporal structure at the cost of requiring finite support in the current implementation.

17.5 Coalgebraic Foundations

Our coinductive stream order $\leq_s$ is directly inspired by coalgebraic semantics for infinite behaviors [10]. While coalgebra has been used

to model MDPs and bisimulation, our use is distinctive in treating optimality itself as a coinductive homomorphism from action rankings to stream orders. The guarded coinduction available in Agda makes the resulting definitions executable and machine-checkable, something that remains rare in coalgebraic approaches to RL.

MDP homomorphisms have also been explored to exploit state/action symmetries for computational efficiency [15]. Our use of "symmetric" is group-theoretic but on a different object—the symmetric group $S_{|\mathcal{A}|}$ acting on the action set rather than automorphisms of the state space.

17.6 Formal Verification of RL

CertRL [14] provides machine-checked convergence proofs for classical value and policy iteration using the Giry monad. Our work is complementary: we verify a different foundation (coinductive structure preservation rather than scalar convergence) and target a broader class of phenomena, including sacrifice patterns and stochastic dominance hierarchies. Both lines of work demonstrate that rigorous, machine-checked RL theory is feasible; they address different parts of the verification stack.

18 Future Work

- **Continuous distributions:** The stochastic framework (§11) represents distributions as `List (A × ℕ)`. Extending to measure-theoretic distributions would broaden applicability to continuous-outcome environments.
- **Continuous action spaces:** The framework assumes finite, enumerable actions. Extending to continuous action spaces (e.g., torque values) would require replacing enumeration-based ranking with optimization over compact sets.
- **Partially observable MDPs:** Lift rankings to belief-state distributions with stochastic transitions.
- **Multi-agent settings:** Rankings over joint action spaces with game-theoretic equilibrium conditions.
- **Program synthesis as policy representation:** The full-ranking objective (Remark 1) naturally extends to synthesized programs: a program satisfying all pairwise ranking constraints on observed states encodes sufficient environment structure to generalize to unseen states.

19 Conclusion

We have presented a framework that redefines optimality as *structure preservation* rather than scalar maximization. Two coinductive conditions capture two complementary notions of optimality:

- **`CoinductiveHomomorphism`** evaluates actions by the quality of the states they lead to—successor state quality. It is the strategic condition: an action that sacrifices immediate reward to reach a superior successor is correctly ranked.
- **`CoindHomo`** additionally requires that immediate transition rewards respect the ranking. It is the tactical condition: the better-ranked action must win both the immediate payoff and the long-term future.

The decomposition theorem establishes their precise relationship: `CoindHomo` equals `CoinductiveHomomorphism` plus immediate reward compatibility. For the vast majority of standard environments, these conditions coincide; three verified counterexamples demonstrate the strict gap.

The framework is supported by machine-verified results in Agda:

1. **Subsumption:** Both conditions imply classical argmax optimality for any finite horizon.
2. **Monotonic learning:** Violation-correction converges in at most $|S| \times \binom{|A|}{2}$ swaps.
3. **Stochastic extension:** The framework lifts to probabilistic environments via the Giry monad, with lexicographic coinductive comparison.
4. **Distributional hierarchy:** FOSD and the SD[k] chain (§11) give parameterized verification strength for progressively broader utility function classes.
5. **Compositional algebra:** Verified rankings compose via product, sum, convolution, and scaling.
6. **State abstraction:** Rankings verified on finite abstract systems lift to arbitrary concrete spaces.
7. **Scalable verification:** The Environment Class architecture provides automatic trace-to-stream bridges; 8-Queens achieves a full `CoindHomo` for 8^8 candidate placements with zero postulates.

All Agda code shown in this paper is machine-checked: the literate source type-checks under Agda while producing this document.

References

1. Bellemare, M.G., Dabney, W., Munos, R.: A distributional perspective on reinforcement learning. In: Proc. ICML, pp. 449–458 (2017)

2. Bellman, R.: Dynamic Programming. Princeton University Press (1957)
3. Dabney, W., Rowland, M., Bellemare, M.G., Munos, R.: Distributional reinforcement learning with quantile regression. In: Proc. AAAI, pp. 2892–2901 (2018)
4. García, J., Fernández, F.: A comprehensive survey on safe reinforcement learning. J. Mach. Learn. Res. 16(42), 1437–1480 (2015)
5. Giry, M.: A categorical approach to probability theory. In: Categorical Aspects of Topology and Analysis. LNM, vol. 915, pp. 68–85. Springer (1982)
6. Hutter, M.: Universal Artificial Intelligence: Sequential Decisions Based on Algorithmic Probability. Springer (2005)
7. Li, L., Walsh, T.J., Littman, M.L.: Towards a unified theory of state abstraction for MDPs. In: Proc. ISAIM (2006)
8. Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., et al.: Human-level control through deep reinforcement learning. Nature 518(7540), 529–533 (2015)
9. Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C.D., Finn, C.: Direct preference optimization: your language model is secretly a reward model. In: Proc. NeurIPS (2023)
10. Rutten, J.J.M.M.: Universal coalgebra: a theory of systems. Theor. Comput. Sci. 249(1), 3–80 (2000)
11. Skalse, J., Hammond, L., Griffin, C., Abate, A.: Lexicographic multi-objective reinforcement learning. In: Proc. IJCAI, pp. 3430–3436 (2022)
12. Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction, 2nd edn. MIT Press (2018)
13. Tercan, H., Prabhu, S.: Thresholded lexicographic ordered multiobjective reinforcement learning. In: Proc. ECAI (2024)
14. Vajjha, K., Shinnar, A., Trager, B., Pestun, V., Fulton, N.: CertRL: formalizing convergence proofs for value and policy iteration in Coq. In: Proc. CPP, pp. 18–31. ACM (2021)
15. van der Pol, E., Worrall, D., van Hoof, H., Oliehoek, F., Welling, M.: MDP homomorphic networks: group symmetries in reinforcement learning. In: Proc. NeurIPS (2020)
16. Watkins, C.J.C.H., Dayan, P.: Q-learning. Mach. Learn. 8(3–4), 279–292 (1992)
17. Wirth, C., Akrour, R., Neumann, G., Fürnkranz, J.: A survey of preference-based reinforcement learning methods. J. Mach. Learn. Res. 18(136), 1–46 (2017)
18. Zap, A., Chaudhuri, S., Topcu, U.: Deep ordinal reinforcement learning. In: Proc. AAMAS, pp. 1838–1846 (2019)